



Security Review For Noon Capital



Collaborative Audit Prepared For: **Noon Capital**
Lead Security Expert(s): **Oxeix**

Kirkeelee

Kvar

Date Audited:

June 16 - June 26, 2026

Introduction

Noon is a yield-bearing stablecoin protocol. It issues USN, a dollar-denominated stablecoin, and sUSN, its staked form, which accrues the returns generated by the protocol's reserves. Governance runs through the NOON token.

Reserves are deployed across a mix of on-chain and off-chain credit. Noon is using collateralised loan obligations, private credit funds, business development companies, DeFi lending markets, and principal-token yield trading. USN and sUSN trade permissionlessly on major DEXes; minting and redemption with the protocol are restricted to whitelisted wallets.

Scope

Repository: [dclif-labs/Governance-Core](#)

Audited Commit: [963dadb3f84c4e95c6e23fdf19ed83267d91671d](#)

Final Commit: [963dadb3f84c4e95c6e23fdf19ed83267d91671d](#)

Files:

- [contracts/stakeNOONRewarder.sol](#)

Repository: [dclif-labs/Protocol-Core](#)

Audited Commit: [fcf78793bd5b167f71600965d06c1da09a6a4b86](#)

Final Commit: [45ee4e1963885696c782bf9cbf0585845695810f](#)

Files:

- [contracts/MinterHandlerV2.sol](#)
- [contracts/periphery/Timelock.sol](#)
- [contracts/RedeemHandlerV2.sol](#)
- [contracts/remote/StakedUSNHyperlane.sol](#)
- [contracts/StakingVaultOFTUpgradeableHyperlane.sol](#)
- [contracts/USNUpgradeableHyperlane.sol](#)

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	6	8

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Global oracle staleness threshold can disable feeds with longer heartbeats [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/8>

Summary

A single one-hour `oracleStalenessThreshold` is applied to every collateral feed, although Chainlink feeds may have different heartbeat intervals.

Vulnerability Detail

If a collateral feed's normal heartbeat exceeds one hour and its deviation threshold is not triggered, its latest price will be considered stale:

```
uint256 public oracleStalenessThreshold = 1 hours;

if (block.timestamp - updatedAt > oracleStalenessThreshold) {
    revert StalePrice(updatedAt, block.timestamp);
}
```

Consequently, a normally functioning slower feed may be rejected. Increasing the global threshold to accommodate that feed would also weaken staleness protection for faster feeds. For example, Chainlink's official USDC/USD Ethereum feed page is indexed with a heartbeat above one hour.

Impact

Direct minting or redemption can be unavailable for collaterals whose oracle heartbeat exceeds the global threshold.

Tool Used

Manual Review

Recommendation

Configure a separate staleness threshold for each collateral feed according to its documented heartbeat:

```
+mapping(address collateral => uint256 maxStaleness)
+    public collateralMaxStaleness;
```

Issue M-2: The protocol can consistently be left with less value due to arbitrage [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/10>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

Minting and redeeming use Chainlink collateral/USD price feeds. The supported collateral assets are expected to be stablecoins.

Chainlink feeds do not update continuously. A feed updates when either its heartbeat elapses or the price movement exceeds the feed's deviation threshold. This means the oracle data can be fresh, but still differ from the current market price. For example, the Ethereum USDC/USD feed has a 0.25% deviation threshold, the Ethereum USDS/USD feed has a 0.3% deviation threshold.

Because of this, the oracle can still report a stablecoin at \$1.00 while its actual market price is slightly below or above \$1.00.

This becomes more problematic because users can mint with one stablecoin and redeem into another. For example, assume USDC is \$0.998 and USDS is \$1.0029, but both Chainlink feeds still report \$1.00.

A user can deposit 1,000,000 USDC, mint 1,000,000 USN, and then redeem that USN into 1,000,000 USDS. In market value, the user deposited around \$998,000 and received around \$1,002,900. For simplicity, this example ignores `priceThresholdBps`. It does not help here, since it checks the Chainlink price, not the live market price.

Impact

Value can be consistently extracted from the protocol through arbitrage.

Recommendation

Add mint and redeem fees. Since users can mint with one collateral and redeem another, the fees should cover deviation on both sides. For example, 0.4% mint fee and 0.4% redeem fee. Fees can go to reserves or be used for rebasing.

Issue M-3: Users can stake right before rebase and receive unearned yield [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/11>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

A user can front-run `mintAndRebase()` by staking into `StakingVault0FTUpgradeableHyperlane` right before the rebase and accrue yield they were not meant to receive.

This is partially mitigated by the USN mint limit, but it does not fully solve the issue. First, the user can still mint USN up to the mint limit and stake that USN before the rebase. Second, the user may already hold USN, so they do not need to mint at all. They can simply stake existing USN when a rebase is about to happen.

Impact

Users front-running a rebase can receive yield they were not meant to get, reducing the yield for existing stakers.

Recommendation

There are a few options: add a deposit/withdraw fee, add a deposit delay, or make new deposits ineligible for the next rebase.

The best fix would be to avoid a stepwise jump in share price and stream/distribute the yield gradually over time instead, though this is a larger change.

Issue M-4: Redemptions can be blocked by cycling `directMint()` and `directRedeem()` [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/13>

Vulnerability Detail

`MinterHandlerV2` has per-block and daily mint limits. `RedeemHandlerV2` also has per-block and daily redeem limits. The redeem limits are expected to be set significantly lower than the mint limits. However, a user can consume the redeem limits by cycling `directMint()` and `directRedeem()`, since there is no protection against this flow.

Consider this:

1. User takes a Balancer flashloan in a collateral that is supported for both minting and redeeming.
2. User calls `directMint()` and mints up to the redeem limit.
3. User calls `directRedeem()` with all minted USN.
4. User receives collateral back and repays the flashloan.

This consumes the per-block redeem limit. The daily redeem limit can also be consumed if the same flow is repeated a few times across separate blocks.

`directMint()` and `directRedeem()` are restricted to KYB'd partners only, but that does not fully mitigate the issue.

Impact

The redeem limits can be consumed and therefore redemptions can be blocked.

Recommendation

There are a few options here:

- add a cooldown before USN minted through `directMint()` can be redeemed
- add a redemption delay, or add mint/redeem fees

We would recommend a short cooldown before freshly minted USN can be redeemed, together with mint/redeem fees as that will mitigate other issues reported as well.

Issue M-5: Whitelist Check in `_update` Blocks All Mints, Burns, and Cross-Chain Transfers [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/14>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The whitelist check at `_update` requires both `from` AND `to` to be whitelisted when `!permissionless`. OpenZeppelin v5.x `_mint` calls `_update(address(0), to, amount)` and `_burn` calls `_update(from, address(0), amount)`. Since `address(0)` is never whitelisted (mapping default is false), every mint, burn, OFT cross-chain send/receive, and Hyperlane send/receive revert before `enablePermissionless()` is called. Same issue exists in `USN.sol` `_update`.

Vulnerability Detail

In `USNUpgradeableHyperlane.sol`, the `_update` override at line 219:

```
if (!permissionless && (!isWhitelisted(from) || !isWhitelisted(to))) revert
↳ NotWhitelisted(from, to);
```

ERC20 `_update` is the single internal entry point for all token movements. OZ v5.x routes `_mint` as `_update(address(0), to, amount)` and `_burn` as `_update(from, address(0), amount)`. Since `address(0)` returns false from `isWhitelisted()`, every operation that passes through `_update` with a zero address fails:

- `mint()` (L177) → `_mint(to, amount)` → `_update(address(0), to, ...)` – `from` is `address(0)`, not whitelisted
- `sendTokensViaHyperlane()` (L99) → `_burn(msg.sender, _amount)` → `_update(from, address(0), ...)` – `to` is `address(0)`, not whitelisted
- `handle()` (L147) → `_mint(recipient, amount)` – same as `mint()`
- **LayerZero OFT send** (inherited `_debit`) → `_burn` → same
- **LayerZero OFT receive** (inherited `_credit`) → `_mint` → same

Impact

The token is completely non-functional in permissioned mode. No mints, burns, or cross-chain transfers can occur until `enablePermissionless()` is called, defeating the purpose of the permissioned phase.

Code Snippet

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/84beff4ed1bfc91821923157faa83ddfa289f6a4/Protocol-Core/contracts/USNUpgradeableHyperlane.sol#L213-L221>

Tool Used

Manual Review

Recommendation

Treat `address(0)` as implicitly allowed, it represents mint (`from=0`) or burn (`to=0`), not a real transfer participant:

```
// USNUpgradeableHyperlane.sol:_update
bool fromOk = from == address(0) || isWhitelisted(from);
bool toOk   = to   == address(0) || isWhitelisted(to);
if (!permissionless && (!fromOk || !toOk))
    revert NotWhitelisted(from, to);
```

Issue M-6: Minting below-peg collateral at \$1 drains healthy collateral [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/18>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`directMint()` treats collateral as worth exactly 1 when the Chainlink price is within the configured peg band. With the default `priceThresholdBps = 100`, any collateral priced between `0.99` and `$1.01` is treated as `$1`. This allows a whitelisted user to mint USN with discounted collateral and redeem that USN for healthier collateral, shifting the depeg loss to the protocol treasury.

Vulnerability Detail

In `MinterHandlerV2`, collateral inside the peg band is minted 1:1:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/main/Protocol-Core/contracts/MinterHandlerV2.sol#L321-323>

```
if (price >= lowerBound && price <= upperBound) {
    // Within threshold: 1:1 mint
    usnAmount = normalizedCollateral;
}
```

The default threshold is:

```
uint256 public priceThresholdBps = 100; // 1%
```

So if a collateral trades at `$0.991`, it is still treated as `$1`. A whitelisted user can do the following:

```
USDT oracle price = $0.991
USDC oracle price = $1.000
```

1. Deposit `100,000` USDT.
Actual value: `99,100` USD.
2. `directMint()` mints `100,000` USN because USDT is inside the peg band.
3. Redeem `100,000` USN for healthy USDC at `1:1`.

4. User receives 100,000 USDC while only depositing collateral worth 99,100 USD.

The protocol loses approximately \$900 of healthy collateral value. This loss is not caused by an oracle failure; the contract intentionally ignores the reported below-peg price because it is inside the configured band.

Impact

A whitelisted user can extract value from the protocol by converting discounted collateral into healthier collateral through USN. This causes undercollateralized USN issuance and drains stronger collateral from the treasury.

Tool Used

Manual Review

Recommendation

Don't round economically adverse prices to \$1. For minting, use the lower of the oracle price and \$1, and, for redeeming, use the higher of the oracle price and \$1.

Discussion

pabloo-noon

Acknowledged :

- We work as this for now with partners and we might lower to 0.1% deviation (already configurable)

Issue L-1: Whitelisted users can withdraw directly but cannot redeem directly [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/9>

Vulnerability Detail

`StakingVaultOFTUpgradeableHyperlane::withdraw()` allows a whitelisted user to withdraw directly. However, `redeem()` does not have the same functionality. It always forces the redemption through `WithdrawalHandler`, so even whitelisted users have their shares queued instead of being able to redeem directly.

Recommendation

Consider allowing whitelisted users to redeem directly as well.

Issue L-2: `directRedeem()` queues the full `usnAmount` even when only a small part exceeds the limit [RE-SOLVED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/12>

Vulnerability Detail

`RedeemHandlerV2::directRedeem()` checks whether `usnAmount` would exceed the redeem limits. If it does, the whole redemption is queued.

The issue is that even if the amount exceeds the limit by only `1 wei`, the full `usnAmount` is queued instead of redeeming the available part and queueing only the excess.

This allows redemptions to be pushed into the queue unnecessarily. For example, if Alice wants to redeem `X USN` and exactly `X USN` is still available under the limit, Bob can front-run her and redeem `1 wei`. Alice's full redemption will now be queued. She can cancel it, but the same thing can happen again when she retries.

Recommendation

Queue only the amount that exceeds the available limit, instead of queueing the full `usnAmount`.

Also, as a separate improvement, consider adding a minimum queue amount. Allowing dust amounts like `1 wei` to be queued is pointless and can create unnecessary queue entries.

Issue L-3: No Sweep Mechanism for Unclaimed Rewards, endRewardDistribution Leaves NOON Stranded [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/15>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

endRewardDistribution() only sets isActive = false. No function exists to withdraw or recycle unclaimed NOON tokens. If the reward program is ever sunset, unclaimed tokens are permanently locked.

Vulnerability Detail

createRewardDistribution (L46-56) always pulls the full totalReward from the owner via transferFrom. When endRewardDistribution() (L82-86) deactivates the distribution, any unclaimed NOON stays in the contract, there is no sweep, no transfer to owner, and no rescueTokens escape hatch:

```
function endRewardDistribution() external onlyOwner {
    require(currentDistribution.isActive, "No active distribution");
    currentDistribution.isActive = false;
    emit RewardDistributionEnded();
}
```

Users who fail to claim before the distribution ends (overlooked the window, lost access to their address, etc.) have their allocation permanently stranded.

Impact

Unclaimed NOON is locked in the contract forever. If the protocol retires the reward program, the tokens can never be recovered.

Code Snippet

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/84beff4ed1bfc91821923157faa83ddfa289f6a4/Governance-Core/contracts/stakeNOONRewarder.sol#L82-L86>

Tool Used

Manual Review

Recommendation

Encoding the leftover into the next distribution's Merkle root and transferring only the delta can be a workaround, but `createRewardDistribution` always pulls the full `totalReward` and requires `totalReward > 0`, so a recycle-only distribution is impossible. The workaround also makes `totalReward` semantically wrong relative to the actual claimable amount, confusing off-chain tooling and subgraphs.

Sweep remaining NOON to the owner in `endRewardDistribution` so it can be re-injected or withdrawn:

```
function endRewardDistribution() external onlyOwner {
    require(currentDistribution.isActive, "No active distribution");
    currentDistribution.isActive = false;

    uint256 remaining = noon.balanceOf(address(this));
    if (remaining > 0) {
        noon.transfer(owner(), remaining);
    }

    emit RewardDistributionEnded();
}
```

Alternatively, permissioned `rescueTokens` function can be implemented.

Discussion

pabloo-noon

Acknowledged :

- We do incremental distributions
- Leftovers are reusable in future distributions

Issue L-4: registerHyperlaneRemoteToken Silently Overwrites Active Registrations, In-Flight Messages Permanently Stranded [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/16>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Calling `registerHyperlaneRemoteToken(domain, newToken)` overwrites `remoteTokens[domain]` without checking if the previous token has messages in-flight. After the overwrite, `handle()` rejects all in-flight messages from the old remote token (`_sender != expectedToken` reverts with `InvalidRemoteToken()`). The Hyperlane Mailbox marks messages as delivered before calling `handle()`, so these messages can never be retried. Source-chain tokens are permanently stranded.

Affects all three Hyperlane contracts: `USNUpgradeableHyperlane`, `StakingVaultOFTUpgradeableHyperlane`, and `StakedUSNHHyperlane`.

Vulnerability Detail

Each contract's `registerHyperlaneRemoteToken` is a blind overwrite – no check for active registration, no two-step process, no in-flight message accounting:

```
// USNUpgradeableHyperlane.sol:84-88 (identical pattern in all three)
function registerHyperlaneRemoteToken(uint32 _domain, bytes32 _remoteToken)
    ↪ external onlyOwner {
    require(_remoteToken != bytes32(0), "Invalid remote token");
    remoteTokens[_domain] = _remoteToken; // silent overwrite
    emit RemoteTokenSet(_domain, _remoteToken);
}
```

The destination `handle()` uses `remoteTokens[_origin]` to verify the sender:

```
bytes32 expectedToken = remoteTokens[_origin];
if (_sender != expectedToken) revert InvalidRemoteToken();
```

If `handle()` reverts, the message is consumed on the protocol level with no retry. Only `StakingVaultOFTUpgradeableHyperlane` has an admin recovery mechanism (`validateExecuteHandleIssue`, with a 48h timelock). `USNUpgradeableHyperlane` and `StakedUSNHHyperlane` have none.

Impact

An admin overwriting a remote token registration (even for a legitimate reason like upgrading the counterpart contract) instantly orphans all in-flight messages from the old token. Source-chain tokens are burned/locked and can never be recovered in two of the three contracts.

Code Snippet

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/84beff4ed1bfc91821923157faa83ddfa289f6a4/Protocol-Core/contracts/USNUpgradeableHyperlane.sol#L84-L88> <https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/84beff4ed1bfc91821923157faa83ddfa289f6a4/Protocol-Core/contracts/StakingVaultOFTUpgradeableHyperlane.sol#L318-L323> <https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/84beff4ed1bfc91821923157faa83ddfa289f6a4/Protocol-Core/contracts/remote/StakedUSNHHyperlane.sol#L90-L95>

Tool Used

Manual Review

Recommendation

Revert if the domain is already registered, forcing a two-step migration:

```
function registerHyperlaneRemoteToken(uint32 _domain, bytes32 _remoteToken)
↪ external onlyOwner {
    require(_remoteToken != bytes32(0), "Invalid remote token");
    require(remoteTokens[_domain] == bytes32(0), "Already registered");
    remoteTokens[_domain] = _remoteToken;
    emit RemoteTokenSet(_domain, _remoteToken);
}
```

For upgrades, add a separate `updateHyperlaneRemoteToken` with a timelock or a pending-accept two-step pattern to allow in-flight messages to settle before the switch.

Issue L-5: Signed redemption path does not normalize Chainlink feed decimals [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/17>

Summary

RedeemHandlerV2 normalizes Chainlink oracle answers in the direct redemption path, but the signed `redeem()` path uses the raw oracle answer and assumes it already has 8 decimals.

Vulnerability Detail

`directRedeem()` uses `_getPrice()`, which reads `oracle.decimals()` and normalizes the answer to 8 decimals:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/main/Protocol-Core/contracts/RedeemHandlerV2.sol#L647-653>

```
uint8 feedDecimals = oracle.decimals();

if (feedDecimals == 8) {
    return uint256(answer);
} else if (feedDecimals < 8) {
    return uint256(answer) * 10 ** (8 - feedDecimals);
} else {
    return uint256(answer) / 10 ** (feedDecimals - 8);
}
```

However, the signed `redeem()` flow uses `_calculateCollateralFromOracle()`, which calls `getCollateralPrice()`. That function returns the raw Chainlink answer without reading or normalizing `decimals()`. As a result, if any configured collateral feed does not use 8 decimals, signed redemptions will calculate the collateral amount incorrectly, while direct redemptions calculate it correctly.

Impact

For feeds with more than 8 decimals, signed redemptions may severely underpay users or round the output toward zero, causing redemptions to revert or become unusable.

Tool Used

Manual Review

Recommendation

Use the same normalized oracle path for both direct and signed redemptions.

Issue L-6: Signed mint tolerance allows minting more USN than deposited collateral [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/20>

Summary

MinterHandlerV2.mint() allows the signed order's collateral amount and USN amount to differ by up to 2% of the larger value. Because the tolerance is symmetric, it permits orders where the user receives more USN than the normalized collateral amount.

Vulnerability Detail

For non-minter users, the signed mint path compares the normalized collateral amount and normalized USN amount:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/main/Protocol-Core/contracts/MinterHandlerV2.sol#L159-173>

```
uint256 difference;
if (normalizedCollateralAmount > normalizedUsnAmount) {
    difference = normalizedCollateralAmount - normalizedUsnAmount;
} else {
    difference = normalizedUsnAmount - normalizedCollateralAmount;
}

// Calculate 2% of the larger amount
uint256 twoPercent = (
    normalizedCollateralAmount > normalizedUsnAmount ? normalizedCollateralAmount :
    ↪ normalizedUsnAmount
) / 50;

if (difference > twoPercent) {
    revert CollateralUsnMismatch(order.collateralAmount, order.usnAmount);
}
```

Since the tolerance is calculated from the larger amount, an order with 100,000 collateral and 102,000 USN passes:

```
difference = 2,000
twoPercent = 102,000 / 50 = 2,040
```

The contract then mints the signed usnAmount and only transfers the signed collateralAmount:

```
usnToken.mint(order.user, order.usnAmount);  
_transferCollateral(order.collateralAddress, order.user, order.collateralAmount);
```

Impact

If MINTER_ROLE executes such orders, the protocol can mint more USN than the amount of collateral received, causing undercollateralized USN issuance of up to approximately 2.04% per signed mint.

Tool Used

Manual Review

Recommendation

Make the tolerance one-sided. For minting, the protocol should not allow usnAmount to exceed the collateral value unless this is an explicit, separately authorized subsidy.

Issue L-7: Missing custodial wallet check can mint USN while USDT is sent to address(0) [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/21>

Summary

custodialWallet starts as address(0), but mint() and directMint() do not check that it has been configured before transferring collateral. For USDT-like tokens that do not reject transfers to address(0), users can mint USN while the protocol receives no collateral.

Vulnerability Detail

setCustodialWallet() rejects zero addresses:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/main/Protocol-Core/contracts/MinterHandlerV2.sol#L87-93>

```
function setCustodialWallet(address _custodialWallet)
    external
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    if (_custodialWallet == address(0)) {
        revert ZeroAddress();
    }

    custodialWallet = _custodialWallet;
    emit CustodialWalletSet(_custodialWallet);
}
```

However, custodialWallet is not set during initialization and defaults to address(0). The collateral transfer helper does not validate it:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/main/Protocol-Core/contracts/MinterHandlerV2.sol#L486-488>

```
function _transferCollateral(
    address collateral,
    address user,
    uint256 amount
) internal {
    IERC20(collateral).safeTransferFrom(user, custodialWallet, amount);
}
```

For standard USDC, this would revert because Circle's implementation checks `to != address(0)` inside `_transfer()`. So with USDC the impact is mainly minting DoS. However, canonical legacy USDT-style code doesn't include a zero-address recipient check in `transferFrom()` and directly adds the transferred amount to `balances[_to]`. Therefore, `transferFrom()` can succeed instead of reverting.

Impact

The protocol can mint USN without receiving collateral, creating undercollateralized USN. The issue is limited to the deployment/configuration window before `custodialWallet` is set.

Tool Used

Manual Review

Recommendation

Require `custodialWallet` to be configured before any collateral transfer.

Issue L-8: ProxyAdmin ownership-transfer comments state 30 days but code uses 2 days [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/issues/22>

Summary

Timelock comments state that ProxyAdmin ownership transfers are protected by a fixed 30-day delay, but the implemented constant is only 2 days.

Vulnerability Detail

The contract defines the ownership-transfer delay as:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/main/Protocol-Core/contracts/periphery/Timelock.sol#L25-26>

```
/// @dev Fixed delay for ProxyAdmin ownership transfer handover.  
uint256 public constant OWNERSHIP_TRANSFER_DELAY = 2 days;
```

This delay is used by `executeProxyAdminOwnershipTransfer()`:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/main/Protocol-Core/contracts/periphery/Timelock.sol#L98-101>

```
uint256 executeAfter = at + OWNERSHIP_TRANSFER_DELAY;  
if (block.timestamp < executeAfter) {  
    revert DelayNotElapsed(opId, executeAfter, block.timestamp);  
}
```

However, the comments describe a 30-day delay:

<https://github.com/sherlock-audit/2026-06-noon-capital-june-16th-2026/blob/main/Protocol-Core/contracts/periphery/Timelock.sol#L163-165>

```
/**  
 * @dev Schedules a ProxyAdmin ownership transfer. Only owner.  
 *      This path is always guarded by a fixed 30-day delay.  
 */
```

Impact

If operators, governance participants, or integrators rely on the comments, they may believe ProxyAdmin ownership handover requires 30 days of notice, while it can actually

be executed after 2 days. Additionally, the value itself can be incorrect (instead of the intended one).

Tool Used

Manual Review

Recommendation

Either update the comments to match the implemented 2-day delay or, if the intended governance requirement is actually 30 days, update the constant.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.